

Understanding Basic Commands for Synthesis using-Yosys

INTRODUCTION:

In this lab, students will learn the basics of Yosys, an open-source framework for RTL synthesis. They will practice translating Verilog RTL designs into optimized gate-level netlists, mapping them onto standard cell libraries, and using essential commands for elaboration, optimization, technology mapping, and visualization. This exercise helps in clearly understanding the synthesis process and its commands.

APPLICATION:

These commands are used to read Verilog files, optimize RTL designs, and map logic to standard cell libraries. They also generate technology-mapped gate-level netlists and allow visualization for verification. These steps form the backbone of RTL-to-GDSII flows, enabling the design of circuits like counters, adders, and multiplexers etc.

OBJECTIVE:

After completing this lab, you will be able to:

- Write Verilog code for the target design.
- Load the RTL design into Yosys for synthesis.
- Perform synthesis to map the design to the target standard cell library
- Understand mapping of the design to standard cells.
- Generate the gate-level netlist and understand its structure.

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

- 1.1: Create a directory for your lab: `mkdir <project_name>`
- 1.2: Open lab directory: `cd <Project_name>`
- 1.3: Create directory for project: `mkdir practice`
- 1.4: Open project directory: `cd practice`

Step 2: Create the input Design file

- 2.1: Open gvim editor for design with command: `gvim mux_2x1.v` // write the Verilog code

RTL Design Code:

```
module mux_2x1 (  
    input a,  
    input b,  
    input sel,  
    output out  
);  
    assign out = sel ? b : a;  
endmodule
```

2.2: Save the file using command: :wq

Step 3:

Invoke Yosys in a new terminal by the following command:

```
yosys
```

Step 4 :

Read Design (Verilog Source File)

```
read_verilog mux_2x1.v //This loads the Verilog RTL source code into Yosys.
```

```
yosys> read_verilog mux.v  
1. Executing Verilog-2005 frontend: mux.v  
Parsing Verilog input from `mux.v' to AST representation.  
Generating RTLIL representation for module `mux_2x1'.  
Successfully finished Verilog frontend.  
yosys> █
```

Step 5 : Define the Top Module and Check Hierarchy

```
hierarchy -check -top mux_2x1 //top module name depends on RTL
```

Step 6 : Translate Processes and Optimize

```
proc; opt; fsm; opt; memory; opt
```

- **proc** – Converts behavioral constructs like always blocks into processes.
- **fsm** – Extracts and optimizes finite state machines.
- **memory** – Translates memory elements if present.
- **opt** – Optimizes design after each transformation.

Step 7 : Technology Mapping

```
# mapping to internal cell library
```

```
techmap; opt
```

Step 8 : Map Flip-Flops to Library Cells

```
# Assigns flip-flops to available library cells.
```

```
dfflibmap -liberty osu018_stdcells.lib
```

Step 9 : Logic Mapping using ABC

```
# mapping logic to mycells.lib
```

```
abc -liberty osu018_stdcells.lib
```

Step 10 : Clean

```
# Clean up unused cells and signals after mapping
```

```
Clean
```

Step 11 : Visualize the Design

```
show
```

Step 12 : Write Netlist

```
# Write the final synthesized gate-level netlist to a new Verilog file
```

```
write_verilog mux_2x1_synth.v
```

Step 13: View Output File (Netlist)

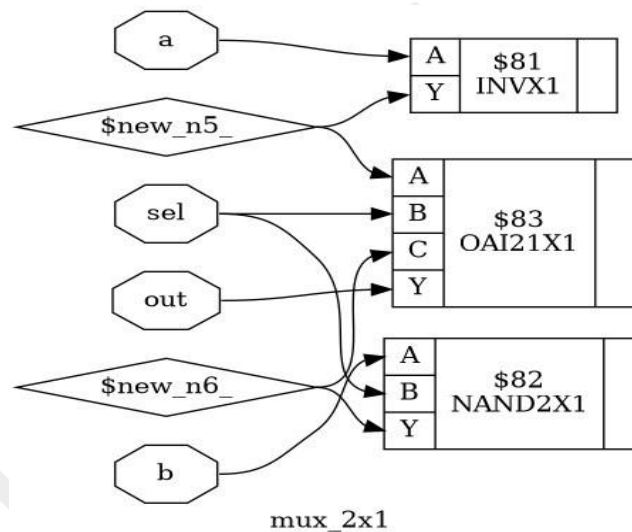
```
gvim mux_2x1_synth.v
```

OBSERVATION:

- Verilog design is successfully read into Yosys.
- Design hierarchy and top-level module are correctly identified.
- Processes, FSMs, and memory blocks are efficiently converted.
- Optimization and technology mapping are applied without errors.
- The synthesized design maps correctly to the OSU018 standard cell library.
- Visualization confirms the expected structure of the design

OUTPUT:

- Synthesized gate-level netlist file.
- Synthesized schematic diagram.



CONCLUSION:

The lab on basic Yosys commands for synthesis demonstrates the complete RTL-to-gate-level flow using a simple Verilog design. Each command plays a vital role, from reading and elaborating the design to optimization, technology mapping, and visualization. The output netlist confirms correct functionality, while the schematic view helps in structural verification. Mastering these commands provides a strong foundation for synthesizing more complex digital circuits and understanding the overall digital design process.